

Heap / Priority Queue — Complete Guide

A comprehensive guide to heaps, priority queues, and all related techniques for placements, online assessments, and competitive programming.

1. Min Heap

1. Overview

A **min heap** is a complete binary tree where the value of every node is **less than or equal to** the values of its children. This means the smallest element in the heap is always at the **root**. A min heap is the underlying data structure for a min-priority queue.

2. Intuition

Simple explanation: Imagine a pyramid of numbers. At the top sits the smallest number. Below it are larger numbers, and below them even larger numbers. If you ever remove the top (smallest) number, the next smallest automatically rises to the top.

Analogy: Think of a company hierarchy where the CEO has the lowest employee ID (smallest number). Every manager reports to someone with a smaller ID. When the CEO leaves, the next smallest ID takes over — and the hierarchy reorganizes itself to maintain the rule.

Step-by-step reasoning:

1. Every parent node must be $\leq$ both its children.
2. The root is always the minimum element.
3. When you insert a new element, place it at the bottom-right and "bubble up" until the heap property is restored.
4. When you delete the root (extract min), replace it with the last element and "bubble down" (heapify) until the property is restored.

3. When to Use It

- You need **fast access to the smallest element** in a dynamic collection.
- You frequently insert and extract the minimum.
- You need to process elements in increasing order but the full sorted list is not needed upfront.

Common trigger phrases:

- "Find the smallest element"
- "Process in increasing order"
- "Kth smallest"
- "Minimum priority"
- "Merge sorted streams"

4. When Not to Use It

- You need **fast access to the largest element** — use a max heap instead.
- You need to search for an arbitrary element — heaps do not support $O(1)$ or $O(\log n)$ search.
- You need **sorted order of all elements** — use sorting ($O(n \log n)$) or a balanced BST.
- The collection is very small ($n < 10$) — a simple array scan is faster and simpler.

5. Core Concepts

Concept	Explanation	Why It Matters
Complete binary tree	All levels are filled except possibly the last, which is left-filled.	Allows compact array representation.
Heap property	Parent $\leq$ children (min heap).	Guarantees root is the minimum.
Bubble up (sift up)	Swap a node with its parent while the heap property is violated.	Restores heap property after insertion.
Bubble down (sift down / heapify)	Swap a node with its smallest child while the heap property is violated.	Restores heap property after deletion or during build.
Array representation	Index $i \rightarrow$ left child = $2i+1$, right child = $2i+2$, parent = $(i-1)/2$.	No pointer overhead; cache-friendly.

6. Step-by-Step Algorithm

Insert:

1. Add the new element at the end of the array (bottom-right of the tree).
2. While the element is smaller than its parent, swap them (bubble up).
3. Stop when heap property is satisfied.

Extract Min (delete root):

1. Save the root value (minimum).
2. Replace root with the last element in the array.
3. Remove the last element.
4. While the new root is larger than either child, swap with the smaller child (bubble down).
5. Return the saved minimum.

Build Heap from array:

1. Start from the last non-leaf node (index $(n/2)-1$).
2. Apply bubble down on each node, going backwards to the root.

7. Dry Run

Insert values: [5, 3, 8, 1, 2] into a min heap

Step	Array (after op)	Tree shape	Root
Insert 5	[5]	5	5
Insert 3	[3, 5]	3→5	3
Insert 8	[3, 5, 8]	3→5,8	3
Insert 1	[1, 3, 8, 5]	1→3,8; 3→5	1
Insert 2	[1, 2, 8, 5, 3]	1→2,8; 2→5,3	1

Extract min twice:

Step	Array	Root extracted
Initial	[1, 2, 8, 5, 3]	—
After extract	[2, 3, 8, 5]	1
After extract	[3, 5, 8]	2

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class MinHeap {
private:
    vector<int> arr;

    int parent(int i) { return (i - 1) / 2; }
    int left(int i) { return 2 * i + 1; }
    int right(int i) { return 2 * i + 2; }

    void bubbleUp(int i) {
        while (i > 0 && arr[parent(i)] > arr[i]) {
            swap(arr[parent(i)], arr[i]);
            i = parent(i);
        }
    }

    void bubbleDown(int i) {
        int n = arr.size();
        while (true) {
            int smallest = i;
            int l = left(i), r = right(i);
            if (l < n && arr[l] < arr[smallest]) smallest = l;
            if (r < n && arr[r] < arr[smallest]) smallest = r;
            if (smallest == i) break;
            swap(arr[i], arr[smallest]);
            i = smallest;
        }
    }
}
```

```

public:
    MinHeap() {}

    MinHeap(const vector<int>& values) {
        arr = values;
        for (int i = (int)arr.size() / 2 - 1; i >= 0; --i)
            bubbleDown(i);
    }

    void push(int val) {
        arr.push_back(val);
        bubbleUp(arr.size() - 1);
    }

    int pop() {
        if (arr.empty()) throw out_of_range("Heap is empty");
        int rootVal = arr[0];
        arr[0] = arr.back();
        arr.pop_back();
        if (!arr.empty()) bubbleDown(0);
        return rootVal;
    }

    int top() {
        if (arr.empty()) throw out_of_range("Heap is empty");
        return arr[0];
    }

    bool empty() { return arr.empty(); }
    int size() { return (int)arr.size(); }
};

// --- Example usage ---
int main() {
    MinHeap h;
    h.push(5); h.push(3); h.push(8); h.push(1); h.push(2);
    while (!h.empty()) {
        cout << h.pop() << " "; // 1 2 3 5 8
    }
    return 0;
}

```

9. Python Implementation

```

class MinHeap:
    def __init__(self, data=None):
        self.arr = data[:] if data else []
        if self.arr:
            for i in range(len(self.arr) // 2 - 1, -1, -1):
                self._bubble_down(i)

```

```
def _parent(self, i):
    return (i - 1) // 2

def _left(self, i):
    return 2 * i + 1

def _right(self, i):
    return 2 * i + 2

def _bubble_up(self, i):
    while i > 0 and self.arr[self._parent(i)] > self.arr[i]:
        self.arr[self._parent(i)], self.arr[i] = self.arr[i],
self.arr[self._parent(i)]
        i = self._parent(i)

def _bubble_down(self, i):
    n = len(self.arr)
    while True:
        smallest = i
        l, r = self._left(i), self._right(i)
        if l < n and self.arr[l] < self.arr[smallest]:
            smallest = l
        if r < n and self.arr[r] < self.arr[smallest]:
            smallest = r
        if smallest == i:
            break
        self.arr[i], self.arr[smallest] = self.arr[smallest], self.arr[i]
        i = smallest

def push(self, val):
    self.arr.append(val)
    self._bubble_up(len(self.arr) - 1)

def pop(self):
    if not self.arr:
        raise IndexError("Heap is empty")
    root = self.arr[0]
    self.arr[0] = self.arr[-1]
    self.arr.pop()
    if self.arr:
        self._bubble_down(0)
    return root

def top(self):
    if not self.arr:
        raise IndexError("Heap is empty")
    return self.arr[0]

def __bool__(self):
    return bool(self.arr)

def __len__(self):
    return len(self.arr)
```

```
# --- Example usage ---
h = MinHeap()
for v in [5, 3, 8, 1, 2]:
    h.push(v)
while h:
    print(h.pop(), end=" ") # 1 2 3 5 8
```

10. Code Explanation

Class structure:

- The heap is stored in a `vector<int>` / list. Index math replaces explicit tree pointers.
- `parent()`, `left()`, `right()` compute indices — this is the core of the array representation.

Bubble up:

- Used after `push()`. The new element goes at the end. We walk it up while it's smaller than its parent.
- Guarantees $O(\log n)$ because the tree height is $\log n$.

Bubble down (heapify):

- Used after `pop()` and during `buildHeap`. The element at index `i` is swapped with its smallest child until the heap property holds.
- Building from the last non-leaf node downwards is $O(n)$ — surprisingly efficient.

Build heap (constructor):

- Starting from $n/2 - 1$ (last parent) and moving left ensures all subtrees already satisfy the heap property when we process a node.
- This is the Floyd build method and runs in $O(n)$, not $O(n \log n)$.

Edge case handling:

- `pop()` and `top()` check for empty heap and throw.
- After `pop()`, `bubbleDown(0)` is only called if the heap is non-empty.

11. Complexity Analysis

Operation	Time	Space	Notes
<code>push()</code>	$O(\log n)$	$O(1)$	Bubble up from leaf
<code>pop()</code>	$O(\log n)$	$O(1)$	Bubble down from root
<code>top()</code>	$O(1)$	$O(1)$	Direct array access
<code>buildHeap()</code>	$O(n)$	$O(1)$ extra	Floyd's method
<code>empty()</code> / <code>size()</code>	$O(1)$	$O(1)$	

Space: $O(n)$ for the array itself.

12. Common Patterns

Pattern	How to Identify	Approach	Example Problems
Kth smallest	"Kth smallest in array/stream"	Use a max heap of size k	Kth Smallest Element in an Array
Running median	"Median of stream"	Two heaps: max heap for left, min heap for right	Find Median from Data Stream
Merge sorted arrays	"Merge k sorted lists"	Min heap of (value, listIndex)	Merge k Sorted Lists
Task scheduling	"CPU tasks", "rearrange tasks"	Max heap by frequency	Task Scheduler, Reorganize String
Top K frequent	"K most frequent elements"	Min heap of size k by frequency	Top K Frequent Elements

13. Common Mistakes

- **Wrong child indices:** $2i+1$ / $2i+2$ (0-based). Using $2i$ / $2i+1$ (1-based) causes out-of-bounds.
- **Infinite loop in bubbleDown:** Not updating `i` after swap — the loop never progresses.
- **Forgetting to check bounds** before accessing children in bubbleDown.
- **Using `>` for min heap or `<` for max heap** incorrectly in comparator.
- **Building heap by inserting one by one** instead of using Floyd's $O(n)$ method when bulk-loading.
- **Modifying the heap while iterating** — leads to undefined behavior.
- **Off-by-one in buildHeap loop:** Start from $n/2 - 1$, not $n/2$.

14. Edge Cases

- **Empty heap:** `pop()` and `top()` must throw or return a sentinel.
- **Single element:** `pop()` should work and leave the heap empty.
- **All equal elements:** Any ordering is valid; heap operations should still maintain $O(\log n)$.
- **Already a heap:** Build should be a no-op; operations should preserve the property.
- **Duplicate values:** Heaps handle duplicates naturally — no special handling needed.
- **Large values:** Use `long long` or 64-bit integers if values can exceed $2^{31}-1$.

15. Variations

Variation	What Changes	When It's Used	Importance
Min Heap	Root is smallest element	Default priority queue in Python	Essential
Max Heap	Root is largest element	C++ <code>priority_queue</code> default; use <code>greater<T></code> for min	Essential
d-ary Heap	Each node has d children	Better cache performance for large heaps; used in some Dijkstra optimizations	CP

Variation	What Changes	When It's Used	Importance
Indexed Heap / Priority Queue with Decrease-Key	Supports updating priority of existing elements	Dijkstra, Prim's algorithm	CP / Placements
Soft Heap	Approximate heap with error parameter	Rare; theoretical interest	Not required

16. Related Algorithms/Data Structures

Structure	Connection	When to Choose
Balanced BST (set/map)	Both maintain sorted order dynamically	BST: need search/delete arbitrary elements. Heap: only need min/max fast
Sorted array + binary search	Can find min in $O(1)$ if sorted	Heap: better for frequent insertions. Array: better for frequent searches
Segment Tree	Can query range min	Segment tree: need range queries. Heap: only need global min
HashSet + Heap	Combined for Top K Frequent pattern	Need both frequency counting and ordering

2. Max Heap

1. Overview

A **max heap** is a complete binary tree where the value of every node is **greater than or equal to** the values of its children. The largest element is always at the root. It's the mirror image of a min heap — every comparison is reversed.

2. Intuition

Simple explanation: The same pyramid, but now the largest number sits on top. Smaller numbers sit below. When you remove the top, the next largest rises.

Analogy: A leaderboard in a game — the highest score is displayed first. When the top player is removed, the next highest moves to the top.

3. When to Use It

- You need **fast access to the largest element** in a dynamic collection.
- C++ `priority_queue` is a max heap by default — use it directly.
- Any "Kth largest" or "top K" problem.

4. When Not to Use It

- You need the smallest element — use a min heap.
- You need sorted order — use `sort()` or a balanced BST.

5. Core Concepts

Same as min heap but with reversed comparison: $\text{parent} \geq \text{children}$.

6. Step-by-Step Algorithm

Same as min heap but:

- Bubble up while the element is **greater** than its parent.
- Bubble down while the element is **smaller** than either child; swap with the **larger** child.

7. Dry Run

Insert [1, 3, 5, 8, 2] into max heap:

Step	Array
Insert 1	[1]
Insert 3	[3, 1]
Insert 5	[5, 1, 3]
Insert 8	[8, 5, 3, 1]
Insert 2	[8, 5, 3, 1, 2]

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class MaxHeap {
private:
    vector<int> arr;

    int parent(int i) { return (i - 1) / 2; }
    int left(int i) { return 2 * i + 1; }
    int right(int i) { return 2 * i + 2; }

    void bubbleUp(int i) {
        while (i > 0 && arr[parent(i)] < arr[i]) {
            swap(arr[parent(i)], arr[i]);
            i = parent(i);
        }
    }

    void bubbleDown(int i) {
        int n = arr.size();
        while (true) {
```

```

        int largest = i;
        int l = left(i), r = right(i);
        if (l < n && arr[l] > arr[largest]) largest = l;
        if (r < n && arr[r] > arr[largest]) largest = r;
        if (largest == i) break;
        swap(arr[i], arr[largest]);
        i = largest;
    }
}

public:
    MaxHeap() {}

    MaxHeap(const vector<int>& values) {
        arr = values;
        for (int i = (int)arr.size() / 2 - 1; i >= 0; --i)
            bubbleDown(i);
    }

    void push(int val) {
        arr.push_back(val);
        bubbleUp(arr.size() - 1);
    }

    int pop() {
        if (arr.empty()) throw out_of_range("Heap is empty");
        int rootVal = arr[0];
        arr[0] = arr.back();
        arr.pop_back();
        if (!arr.empty()) bubbleDown(0);
        return rootVal;
    }

    int top() {
        if (arr.empty()) throw out_of_range("Heap is empty");
        return arr[0];
    }

    bool empty() { return arr.empty(); }
    int size() { return (int)arr.size(); }
};

```

9. Python Implementation

```

import heapq

# Python's heapq is a MIN heap. For max heap, push negative values.
class MaxHeap:
    def __init__(self, data=None):
        self._data = [-x for x in data] if data else []
        heapq.heapify(self._data)

```

```

def push(self, val):
    heapq.heappush(self._data, -val)

def pop(self):
    return -heapq.heappop(self._data)

def top(self):
    return -self._data[0]

def __bool__(self):
    return bool(self._data)

def __len__(self):
    return len(self._data)

# --- Example usage ---
h = MaxHeap()
for v in [1, 3, 5, 8, 2]:
    h.push(v)
while h:
    print(h.pop(), end=" ") # 8 5 3 2 1

```

10. Code Explanation

The max heap is identical to min heap except all comparisons are reversed:

- **bubbleUp**: swap while `arr[parent(i)] < arr[i]` (child is larger).
- **bubbleDown**: swap with the **larger** child, not the smaller.
- Python wrap: negate values because `heapq` is a min-heap.

11. Complexity Analysis

Operation	Time	Space
<code>push()</code>	$O(\log n)$	$O(1)$
<code>pop()</code>	$O(\log n)$	$O(1)$
<code>top()</code>	$O(1)$	$O(1)$
<code>buildHeap()</code>	$O(n)$	$O(1)$ extra

12–16. (See Min Heap sections — identical patterns with reversed comparisons)

3. Heapify

1. Overview

Heapify is the process of converting an arbitrary binary tree (usually stored as an array) into a heap that satisfies the heap property. There are two types: **bubble down** (top-down) and **bubble up** (bottom-up). The term usually refers to the efficient $O(n)$ bottom-up build.

2. Intuition

Simple explanation: You have a messy pile of numbers arranged in a tree. Heapify tidies it up so every parent is $\leq$ its children (min heap) or $\geq$ its children (max heap).

Analogy: Imagine organizing a tournament bracket. You start from the bottom-most matches and work up — ensuring that in each match, the "winner" (smaller/larger number) moves up. By the time you reach the top, the overall winner is at the root.

Why the $O(n)$ method works: Most nodes are near the bottom of the tree, so they only need a small number of swaps. The number of nodes at height h is about $n/2^{(h+1)}$, and each needs at most h swaps. The sum of $h \cdot n/2^{(h+1)}$ over all h converges to $O(n)$.

3. When to Use It

- You need to build a heap from an existing array quickly.
- After modifying a single node (like replacing the root), you heapify that node down.
- Efficient construction before running heap-related algorithms.

4. When Not to Use It

- You're inserting elements one at a time anyway — push is $O(\log n)$ each, total $O(n \log n)$.
- The array is tiny — the constant factor difference doesn't matter.

5. Core Concepts

Concept	Explanation
Bubble down (sift down)	Push a node downward until heap property holds. Used in buildHeap and after pop.
Bubble up (sift up)	Push a node upward until heap property holds. Used after push.
Last non-leaf node	Index $(n/2) - 1$ (0-based). All nodes after this are leaves (already satisfy heap property individually).
Floyd's buildHeap	Start from last non-leaf, bubble down each node going backwards to root. $O(n)$.

6. Step-by-Step Algorithm

Floyd's buildHeap (min heap):

1. Start with index $i = (n/2) - 1$ (last parent).
2. Call `bubbleDown(i)`.
3. Decrement i .
4. Repeat until $i < 0$.

7. Dry Run

Array: [3, 1, 6, 5, 2, 4] → build min heap

$n = 6$, last non-leaf = $(6/2)-1 = 2$ (value 6)

```
Initial:      3
             / \
            1  6
           / \ /
          5  2 4

Step 1: i=2, bubbleDown(6):  3
                             / \
                            1  4
                           / \ /
                          5  2 6

Step 2: i=1, bubbleDown(1):  3
                             / \
                            1  4
                           / \ /
                          5  2 6
                (1 ≤ 5 and 2, no swap)

Step 3: i=0, bubbleDown(3):  1
                             / \
                            3  4
                           / \ /
                          5  2 6
```

Final heap: [1, 3, 4, 5, 2, 6]

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

void heapify(vector<int>& arr, int n, int i) {
    // Bubble down for min heap
    int smallest = i;
    int l = 2 * i + 1, r = 2 * i + 2;

    if (l < n && arr[l] < arr[smallest]) smallest = l;
    if (r < n && arr[r] < arr[smallest]) smallest = r;

    if (smallest != i) {
        swap(arr[i], arr[smallest]);
        heapify(arr, n, smallest);
    }
}
```

```

}

void buildMinHeap(vector<int>& arr) {
    int n = arr.size();
    for (int i = n / 2 - 1; i >= 0; --i) {
        heapify(arr, n, i);
    }
}

// Max heap version
void heapifyMax(vector<int>& arr, int n, int i) {
    int largest = i;
    int l = 2 * i + 1, r = 2 * i + 2;

    if (l < n && arr[l] > arr[largest]) largest = l;
    if (r < n && arr[r] > arr[largest]) largest = r;

    if (largest != i) {
        swap(arr[i], arr[largest]);
        heapifyMax(arr, n, largest);
    }
}

```

9. Python Implementation

```

def heapify(arr, n, i):
    """Bubble down for min heap (in-place)"""
    smallest = i
    l, r = 2 * i + 1, 2 * i + 2

    if l < n and arr[l] < arr[smallest]:
        smallest = l
    if r < n and arr[r] < arr[smallest]:
        smallest = r

    if smallest != i:
        arr[i], arr[smallest] = arr[smallest], arr[i]
        heapify(arr, n, smallest)

def build_min_heap(arr):
    n = len(arr)
    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)
    return arr

# --- Example ---
arr = [3, 1, 6, 5, 2, 4]

```

```
build_min_heap(arr)
print(arr)  # [1, 3, 4, 5, 2, 6]
```

10. Code Explanation

- `heapify(arr, n, i)` assumes that the left and right subtrees of `i` are already valid heaps. It only fixes node `i`.
- The `buildMinHeap` loop goes from the last parent down to 0. This guarantees that when processing node `i`, both its children (which have larger indices) are already valid heaps.
- Recursive or iterative `bubbleDown` — both work. The iterative version avoids recursion depth issues.

11. Complexity Analysis

Method	Time	Space
Floyd's buildHeap	$O(n)$	$O(1)$ extra ($O(\log n)$ if recursive)
Insert one-by-one	$O(n \log n)$	$O(1)$ extra

Why $O(n)$? The number of nodes at height h is $\leq n/2^{(h+1)}$. Each node at height h needs at most h swaps.
 $\text{Sum} = \sum h \cdot n/2^{(h+1)} \approx n$.

12–16. (See Min Heap — same concepts apply to heapify directly)

4. Priority Queue

1. Overview

A **priority queue** is an abstract data type where each element has a priority. Elements are dequeued in order of their priority — highest (or lowest) priority first, regardless of insertion order. A heap is the most common implementation, but a priority queue can also be implemented with a balanced BST or a skip list.

2. Intuition

Simple explanation: A regular queue is FIFO — first come, first served. A priority queue is like an emergency room waiting list — the most critical patient (highest priority) is treated first, even if they arrived later.

Analogy: An airport boarding system where first-class passengers board before economy, regardless of when they arrived at the gate. Within the same class, it's FIFO (but pure priority queues don't guarantee FIFO among equal-priority items unless stable).

3. When to Use It

- Any problem that says "priority", "schedule", "process by importance".
- Dijkstra's shortest path, Prim's MST, Huffman coding, A* search.
- Task scheduling, event simulation, load balancing.
- Merging sorted streams, finding Kth largest/smallest.

4. When Not to Use It

- You only need a simple queue (FIFO) — use `queue` / `deque`.
- You only need a stack (LIFO) — use `stack`.
- You need to access or search arbitrary elements — use a `set` / `map`.
- The number of elements is very small — linear scan is simpler.

5. Core Concepts

Concept	Explanation
Abstract Data Type (ADT)	Priority queue is a concept, not a specific implementation. Heap is one implementation.
Min-priority queue	Smallest priority is dequeued first.
Max-priority queue	Largest priority is dequeued first.
Stability	Whether equal-priority elements preserve insertion order. Heap-based PQs are not stable.
Decrease-key	Change the priority of an existing element. Required in Dijkstra's algorithm for optimal performance.

6. Step-by-Step Algorithm

Using C++ `priority_queue` (max heap by default):

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    // Max heap (default)
    priority_queue<int> pq;
    pq.push(5); pq.push(1); pq.push(3);
    cout << pq.top(); // 5
    pq.pop();
    cout << pq.top(); // 3

    // Min heap
    priority_queue<int, vector<int>, greater<int>> minPq;

    // Custom comparator
    auto cmp = [](int a, int b) { return a > b; }; // min heap
    priority_queue<int, vector<int>, decltype(cmp)> customPq(cmp);

    return 0;
}
```

7. Dry Run

Operations: push(5), push(2), push(8), push(1), pop(), pop()

State (max heap):

push(5) → [5]

push(2) → [5, 2]

push(8) → [8, 2, 5]

push(1) → [8, 2, 5, 1]

pop() → returns 8, heap: [5, 2, 1]

pop() → returns 5, heap: [2, 1]

8. C++ Implementation (using STL)

```
#include <bits/stdc++.h>
using namespace std;

// Priority queue is already in STL.
// The only things you need to know:

// 1. Max heap (default)
priority_queue<int> pq;

// 2. Min heap
priority_queue<int, vector<int>, greater<int>> minPq;

// 3. Custom comparator
struct Compare {
    bool operator()(int a, int b) {
        return a > b; // true → a has lower priority than b → min heap
    }
};
priority_queue<int, vector<int>, Compare> customPq;

// 4. Priority queue of pairs (sort by first, then second)
using pii = pair<int, int>;
priority_queue<pii, vector<pii>, greater<pii>> minPairPq;
```

9. Python Implementation

```
import heapq

# heapq is a MIN heap.
h = []
heapq.heappush(h, 5)
heapq.heappush(h, 2)
heapq.heappush(h, 8)
heapq.heappush(h, 1)
```

```

print(heapq.heappop(h)) # 1
print(heapq.heappop(h)) # 2

# Max heap using negative values
h_max = []
for v in [5, 2, 8, 1]:
    heapq.heappush(h_max, -v)

print(-heapq.heappop(h_max)) # 8
print(-heapq.heappop(h_max)) # 5

# Priority queue of tuples (priority, item)
pq = []
heapq.heappush(pq, (3, "low"))
heapq.heappush(pq, (1, "high"))
heapq.heappush(pq, (2, "medium"))
print(heapq.heappop(pq)[1]) # "high"

```

10. Code Explanation

C++ STL `priority_queue`:

- Template args: `priority_queue<T, Container, Compare>`.
- Default: `vector<T>` container, `less<T>` comparator → max heap.
- For min heap: use `greater<T>` as comparator.
- Custom comparator must define `operator()` returning `true` if first arg should be placed **after** second.

Python `heapq`:

- `heapq` works on a regular list; there is no separate PQ class.
- `heapq.heappush(list, item)` and `heapq.heappop(list)`.
- For max heap: negate values.
- For complex priorities: push tuples `(priority, value)`.

11. Complexity Analysis

Operation	Time
<code>push()</code>	$O(\log n)$
<code>pop()</code>	$O(\log n)$
<code>top()</code>	$O(1)$
<code>size()</code>	$O(1)$

Space: $O(n)$

12. Common Patterns

Pattern	Approach
K largest	Min heap of size k; push all, pop if size > k
K smallest	Max heap of size k
Schedule tasks	Max heap by frequency; pop, process, decrement count
Median from stream	Two heaps (max + min)
Dijkstra	Min heap of (distance, node)

13. Common Mistakes

- **Incorrect comparator direction** — C++: `greater<T>` for min heap; Python: push negative for max.
- **Modifying elements after pushing** — Priority queue does not re-heapify on mutation.
- **Not remembering that `pq.top()` returns const reference** and modifying it breaks the heap.
- **Using PQ when you need decrease-key** — standard STL PQ doesn't support it; use `set` or indexed heap.

14. Edge Cases

- **Empty PQ**: `top()` and `pop()` on empty — undefined behavior in C++, `IndexError` in Python.
- **Duplicate priorities**: PQ handles them fine; no guaranteed order among equals.
- **Single element**: Works correctly.
- **All equal priorities**: Effectively a queue (FIFO-ish, but not guaranteed).

15. Variations

Variation	Description
Fibonacci Heap	$O(1)$ amortized insert, decrease-key; $O(\log n)$ extract-min. Important for theoretical optimal Dijkstra.
Pairing Heap	Simpler than Fibonacci, good practical performance.
Binomial Heap	Supports fast merge of two heaps.
Indexed Priority Queue	Supports decrease-key via a map from element to index.

16. Related Data Structures

Structure	Use Case
Queue	FIFO, no priority needed
Stack	LIFO
Set / Map	Need ordered traversal, search, or delete arbitrary elements
Deque	Double-ended operations

5. K Largest / K Smallest Elements

1. Overview

Finding the **K largest** or **K smallest** elements in an array is one of the most common heap interview problems. It can be solved in $O(n \log k)$ time using a heap of size k .

2. Intuition

Simple explanation: To find the K largest elements, maintain a min heap of size K . Every time you see a new element, if it's larger than the smallest in the heap (the root), remove the root and add this new element. At the end, the heap contains the K largest elements.

Why this works: The min heap keeps the smallest of the K largest candidates at the root. Any element smaller than that can never be in the top K , so it's safe to ignore.

Analogy: Imagine you're building a "hall of fame" that can hold only K people. When a new person arrives, they must beat the weakest person currently in the hall. If they do, the weakest leaves and the newcomer enters. After processing everyone, the hall holds the K strongest.

3. When to Use It

- "Kth largest", "Kth smallest", "top K", "K closest", "K most frequent".
- Streaming data where you can't store everything.
- Large arrays where sorting $O(n \log n)$ is too expensive.

4. When Not to Use It

- K is close to n : sorting $O(n \log n)$ may be simpler and equally fast.
- You need the K th element only (not the full top K): **quickselect** ($O(n)$ average) is faster.
- K is 1: just scan for min/max in $O(n)$.

5. Core Concepts

Concept	Explanation
Min heap of size K	For finding K largest elements. The root is the smallest among the K largest so far.
Max heap of size K	For finding K smallest elements. The root is the largest among the K smallest so far.
Threshold	The root of the heap acts as a dynamic threshold — only elements beating it enter the top K .

6. Step-by-Step Algorithm

K largest elements:

1. Create a min heap.

- 2. For each element in the array:
 - Push it into the heap.
 - If heap size > K, pop the smallest.
- 3. After processing all elements, the heap contains the K largest elements.

K smallest elements:

- 1. Create a max heap (or min heap with negation in Python).
- 2. For each element:
 - Push it.
 - If heap size > K, pop the largest.
- 3. The heap contains the K smallest elements.

7. Dry Run

Find top 3 largest in [3, 1, 5, 12, 2, 11]

Element	Heap (min heap of size 3)	Action
3	[3]	push
1	[1, 3]	push
5	[3, 5, 1] → [1, 3, 5]	push
12	[1, 3, 5] → [3, 5, 12]	push, pop 1
2	[3, 5, 12]	2 < root(3), skip
11	[3, 5, 12] → [5, 11, 12]	push, pop 3

Final heap: [5, 11, 12] — the 3 largest elements.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

vector<int> findKLargest(vector<int>& nums, int k) {
    if (k <= 0) return {};
    // Min heap
    priority_queue<int, vector<int>, greater<int>> minHeap;
    for (int x : nums) {
        minHeap.push(x);
        if ((int)minHeap.size() > k)
            minHeap.pop();
    }
    vector<int> result;
    while (!minHeap.empty()) {
        result.push_back(minHeap.top());
        minHeap.pop();
    }
    reverse(result.begin(), result.end()); // descending order
}
```

```

        return result;
    }

    vector<int> findKSmallest(vector<int>& nums, int k) {
        if (k <= 0) return {};
        // Max heap
        priority_queue<int> maxHeap;
        for (int x : nums) {
            maxHeap.push(x);
            if ((int)maxHeap.size() > k)
                maxHeap.pop();
        }
        vector<int> result;
        while (!maxHeap.empty()) {
            result.push_back(maxHeap.top());
            maxHeap.pop();
        }
        // Result is in descending order (largest among K smallest first)
        // To get ascending: reverse
        return result;
    }

```

9. Python Implementation

```

import heapq

def find_k_largest(nums, k):
    """Return the K largest elements in nums."""
    if k <= 0:
        return []
    min_heap = []
    for x in nums:
        heapq.heappush(min_heap, x)
        if len(min_heap) > k:
            heapq.heappop(min_heap)
    return sorted(min_heap, reverse=True)

def find_k_smallest(nums, k):
    """Return the K smallest elements in nums."""
    if k <= 0:
        return []
    max_heap = [] # store negatives
    for x in nums:
        heapq.heappush(max_heap, -x)
        if len(max_heap) > k:
            heapq.heappop(max_heap)
    return sorted(-x for x in max_heap)

# --- Example ---

```

```
nums = [3, 1, 5, 12, 2, 11]
print(find_k_largest(nums, 3)) # [12, 11, 5]
print(find_k_smallest(nums, 3)) # [1, 2, 3]
```

10. Code Explanation

- A **min heap of size k** keeps the K largest: the smallest of those K is at the root, and any element $\leq$ root can't be in top K.
- Condition `if (minHeap.size() > k) minHeap.pop()` ensures size never exceeds K.
- The K largest elements in the heap are *not* sorted. We optionally sort at the end.
- For K smallest, use a **max heap** — the largest among the K smallest is at the root. Any element $\geq$ root can't be in the bottom K.

11. Complexity Analysis

Approach	Time	Space
Heap (size K)	$O(n \log K)$	$O(K)$
Quickselect	$O(n)$ average, $O(n^2)$ worst	$O(1)$
Sort	$O(n \log n)$	$O(1)$

12. Common Patterns

Pattern	Approach
Kth largest element	Min heap of size K; answer is root
K closest to origin	Max heap of size K by distance
K most frequent	HashMap + min heap of size K by frequency
K smallest in sorted matrix	Min heap + merge pattern

13. Common Mistakes

- **Using max heap for K largest** — you'd keep removing the largest, which removes what you want to keep.
- **K = 0 or K > n**: Must handle these edge cases.
- **Not comparing correctly after heap size reaches K**: The optimized approach compares before pushing only if needed (though always push-then-pop-if-too-big is simpler and safe).

14. Edge Cases

- $K = 0$: return empty.
- $K \geq n$: return all elements (possibly sorted).
- All equal elements: heap just fills with the same value.
- Negative numbers: works fine — comparisons are value-based.

15. Variations

Variation	Change
Kth largest (not the K set)	Just return the root of the min heap after processing
K closest points to origin	Same pattern, comparator by distance
K largest in stream	Same heap approach works for infinite streams

16. Related

- **Sorting** — simpler but $O(n \log n)$ vs $O(n \log k)$.
- **Quickselect** — $O(n)$ average but not stable and doesn't work well for streams.

6. Merge K Sorted Lists

1. Overview

Given K sorted linked lists (or arrays), merge them into one sorted list. The heap approach runs in $O(N \log K)$ where N is the total number of elements and K is the number of lists.

2. Intuition

Simple explanation: Take the first element from each of the K lists. The smallest among them must be the overall smallest element. Output it, then take the next element from the same list, and repeat.

Analogy: You have K sorted piles of cards face up. At each step, pick the smallest visible card from any pile. The "pile indicator" is a pointer into each list.

Why a heap? Finding the minimum among K candidates takes $O(K)$ time if you scan linearly. A heap reduces that to $O(\log K)$. With N total elements, the heap approach costs $O(N \log K)$ vs $O(NK)$ for linear scanning.

3. When to Use It

- "Merge K sorted lists/arrays/streams".
- "Smallest range covering elements from K lists".
- "K-way merge" problems.

4. When Not to Use It

- Only 2 lists: the standard two-pointer merge is $O(N)$ with $O(1)$ extra space — heap is overkill.
- Lists are not sorted: heap can't help; sort individually first, or just concatenate and sort.

5. Core Concepts

Concept	Explanation
Iterator per list	A pointer tracking the current position in each list.
Min heap of (value, listIndex)	Always gives the smallest current element across all lists.

Concept	Explanation
Advance pointer	After consuming the smallest, move that list's pointer forward.

6. Step-by-Step Algorithm

1. Create a min heap of tuples (value, listIndex).
2. Push the first element of each list into the heap.
3. While the heap is not empty:
 - Pop the smallest (value, listIndex).
 - Append value to result.
 - If the listIndex-th list has a next element, push it into the heap.
4. Return the merged result.

7. Dry Run

Lists: [1, 4, 5], [1, 3, 4], [2, 6]

Step	Heap (value, list)	Output
Init	(1,0), (1,1), (2,2)	[]
Pop (1,0)	(1,1), (2,2)	[1]; push 4 from list 0 → (1,1), (2,2), (4,0)
Pop (1,1)	(2,2), (4,0)	[1, 1]; push 3 from list 1 → (2,2), (3,1), (4,0)
Pop (2,2)	(3,1), (4,0)	[1, 1, 2]; push 6 from list 2 → (3,1), (4,0), (6,2)
Pop (3,1)	(4,0), (6,2)	[1, 1, 2, 3]; push 4 from list 1 → (4,0), (4,1), (6,2)
Pop (4,0)	(4,1), (6,2)	[1, 1, 2, 3, 4]; list 0 exhausted
Pop (4,1)	(6,2)	[1, 1, 2, 3, 4, 4]; list 1 exhausted
Pop (6,2)	empty	[1, 1, 2, 3, 4, 4, 6]

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

struct ListNode {
    int val;
    ListNode *next;
    ListNode(int x) : val(x), next(nullptr) {}
};

struct Compare {
    bool operator()(ListNode* a, ListNode* b) {
        return a->val > b->val; // min heap based on node value
    }
};
```

```

ListNode* mergeKLists(vector<ListNode*>& lists) {
    priority_queue<ListNode*, vector<ListNode*>, Compare> minHeap;

    // Push first node of each list
    for (ListNode* list : lists) {
        if (list) minHeap.push(list);
    }

    ListNode dummy(0);
    ListNode* tail = &dummy;

    while (!minHeap.empty()) {
        ListNode* smallest = minHeap.top();
        minHeap.pop();
        tail->next = smallest;
        tail = tail->next;
        if (smallest->next)
            minHeap.push(smallest->next);
    }

    return dummy.next;
}

```

9. Python Implementation

```

import heapq
from typing import List, Optional

class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Wrapper:
    """Wrapper to make ListNode comparable in heapq (which compares tuples)."""
    def __init__(self, node: ListNode, list_idx: int):
        self.node = node
        self.list_idx = list_idx

    def __lt__(self, other):
        return self.node.val < other.node.val

def merge_k_lists(lists: List[Optional[ListNode]]) -> Optional[ListNode]:
    min_heap = []

    for i, lst in enumerate(lists):
        if lst:
            heapq.heappush(min_heap, (lst.val, i, lst))

    dummy = ListNode(0)

```

```

tail = dummy

while min_heap:
    val, idx, node = heapq.heappop(min_heap)
    tail.next = node
    tail = tail.next
    if node.next:
        heapq.heappush(min_heap, (node.next.val, idx, node.next))

return dummy.next

# --- For merging K sorted arrays instead ---
def merge_k_arrays(arrays):
    """arrays is a list of sorted lists"""
    min_heap = []
    for i, arr in enumerate(arrays):
        if arr:
            heapq.heappush(min_heap, (arr[0], i, 0))

    result = []
    while min_heap:
        val, arr_idx, elem_idx = heapq.heappop(min_heap)
        result.append(val)
        if elem_idx + 1 < len(arrays[arr_idx]):
            next_val = arrays[arr_idx][elem_idx + 1]
            heapq.heappush(min_heap, (next_val, arr_idx, elem_idx + 1))

    return result

```

10. Code Explanation

- **C++:** We use a custom comparator `Compare` for `ListNode*`. The comparator returns `a->val > b->val` to create a min heap (C++ default is max heap).
- **Python:** `heapq` can't directly compare `ListNode` objects. We push tuples `(value, listIndex, node)`. The tuple comparison uses value first, then list index as tiebreaker (to avoid comparing nodes).
- We keep a `tail` pointer that builds the result list.
- After popping a node, we immediately push its next node if it exists.

11. Complexity Analysis

Metric	Complexity
Time	$O(N \log K)$ — N total elements, K lists
Space	$O(K)$ for the heap (+ $O(N)$ for output if counting)

12. Common Patterns

Pattern	Approach
Merge K sorted lists	Heap of (value, listIndex, node)
Smallest range covering K lists	Heap + sliding window
K smallest elements in sorted matrix	Heap of (value, row, col)

13. Common Mistakes

- **Pushing all elements initially** — only push the first from each list; push more as you consume.
- **Not handling null/empty lists** — skip them when initializing.
- **Tiebreaker for equal values** — if two lists have the same value, tuple comparison in Python uses the next tuple element, which is fine.

14. Edge Cases

- $K = 0$: return nullptr/None.
- All lists empty: return nullptr/None.
- Lists of different lengths: handle naturally by checking `node->next` / `node.next`.
- One list very long: heap stays small (size K).

15. Variations

Variation	Change
K sorted arrays	Store array index + element index instead of linked list node
K sorted iterators	Same approach with generic iterators
External merge sort	Merge sorted chunks on disk

16. Related

- **Two-pointer merge** — use for 2 lists, $O(N)$ time, $O(1)$ space.
- **Divide & conquer merge** — merge pairs recursively, $O(N \log K)$ but with $O(1)$ heap vs $O(K)$ heap space.

7. Running Median (Median from Data Stream)

1. Overview

Find the median of a stream of numbers as they arrive one by one. The median is the middle element of a sorted list. If the count is even, the median is the average of the two middle elements.

2. Intuition

Simple explanation: Maintain two halves of the data: a max heap for the smaller half and a min heap for the larger half. The median is always at the top of one of these heaps (or the average of both tops).

Analogy: Imagine a balanced scale. The left side (max heap) holds the smaller numbers, the right side (min heap) holds the larger numbers. The median is the weight at the balance point. We keep the two sides as balanced in size as possible.

Why two heaps? If we kept all numbers in a single sorted list, inserting each new number would be $O(n)$. With two heaps, each insertion is $O(\log n)$, and median retrieval is $O(1)$.

3. When to Use It

- "Find median from data stream".
- "Running median", "sliding window median".
- Any problem where data arrives incrementally and you need the middle value.

4. When Not to Use It

- The full data is known upfront: just sort and take the middle.
- You only need the median once: use quickselect $O(n)$.
- The stream is small: maintain a sorted list with binary insertion.

5. Core Concepts

Concept	Explanation
Max heap (left half)	Stores the smaller half of numbers. The root is the largest in the left half.
Min heap (right half)	Stores the larger half of numbers. The root is the smallest in the right half.
Balance invariant	Size of left heap = size of right heap OR size of left = size of right + 1 (so left has the median when odd).

6. Step-by-Step Algorithm

1. Create a max heap `left` and a min heap `right`.
2. For each new number `x`:
 - If `left` is empty or `x ≤ left.top()`: push `x` into `left`.
 - Else: push `x` into `right`.
 - Rebalance:
 - If `left.size() > right.size() + 1`: move `left.top()` to `right`.
 - If `right.size() > left.size()`: move `right.top()` to `left`.
3. Median:
 - If total elements are odd: median = `left.top()`.
 - If total elements are even: median = $(\text{left.top()} + \text{right.top()}) / 2.0$.

7. Dry Run

Stream: [5, 2, 8, 1, 9]

Number	Left (max heap)	Right (min heap)	Median
5	[5]	[]	5
2	[2, 5] → [5, 2]	[]	5 (rebalance: left size 2 > right+1? No)
Actually: push 2 to left. left=[5,2], right=[]			
left.size=2 > right.size+1? 2 > 1? Yes → move 5			
left=[2], right=[5]			5 → wait, left has [2], right has [5]
			median = 5
8	8 > left.top()=2 → push right: right=[5,8]		
rebal: right.size=2 > left.size=1 → move 5 → left=[2,5], right=[8]			
			median = 5
1	1 ≤ left.top()=5 → push left: left=[2,5,1] → [5,2,1]		
rebal: left.size=3 > right.size+1? 3 > 2? Yes → move 5 → left=[2,1], right=[5,8]			
			median = 2
9	9 > left.top()=2 → push right: right=[5,8,9]		
rebal: right.size=3 > left.size=2? Yes → move 5 → left=[2,1,5]→[5,2,1], right=[8,9]			
			median = 5

Correct medians: 5, 5, 5, 2, 5

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class MedianFinder {
private:
    // Max heap for left half (smaller numbers)
    priority_queue<int> left;
    // Min heap for right half (larger numbers)
    priority_queue<int, vector<int>, greater<int>> right;

public:
    void addNum(int num) {
        // Step 1: Insert into appropriate heap
```

```

        if (left.empty() || num <= left.top()) {
            left.push(num);
        } else {
            right.push(num);
        }

        // Step 2: Rebalance
        if (left.size() > right.size() + 1) {
            right.push(left.top());
            left.pop();
        } else if (right.size() > left.size()) {
            left.push(right.top());
            right.pop();
        }
    }

    double findMedian() {
        if (left.size() > right.size()) {
            return (double)left.top();
        } else {
            return (left.top() + right.top()) / 2.0;
        }
    }
};

// --- Example usage ---
int main() {
    MedianFinder mf;
    vector<int> stream = {5, 2, 8, 1, 9};
    for (int x : stream) {
        mf.addNum(x);
        cout << "Added " << x << ", median: " << mf.findMedian() << "\n";
    }
    return 0;
}

```

9. Python Implementation

```

import heapq

class MedianFinder:
    def __init__(self):
        # Max heap for left half (store negatives)
        self.left = [] # max heap
        # Min heap for right half
        self.right = [] # min heap

    def addNum(self, num: int) -> None:
        # Step 1: Insert into appropriate heap
        if not self.left or num <= -self.left[0]:
            heapq.heappush(self.left, -num)

```

```

        else:
            heapq.heappush(self.right, num)

        # Step 2: Rebalance
        if len(self.left) > len(self.right) + 1:
            heapq.heappush(self.right, -heapq.heappop(self.left))
        elif len(self.right) > len(self.left):
            heapq.heappush(self.left, -heapq.heappop(self.right))

    def findMedian(self) -> float:
        if len(self.left) > len(self.right):
            return -self.left[0]
        else:
            return (-self.left[0] + self.right[0]) / 2.0

# --- Example ---
mf = MedianFinder()
for x in [5, 2, 8, 1, 9]:
    mf.addNum(x)
    print(f"Added {x}, median: {mf.findMedian()}")

```

10. Code Explanation

- **left (max heap):** Stores the smaller half. Its root is the maximum of the smaller half.
- **right (min heap):** Stores the larger half. Its root is the minimum of the larger half.
- **Insertion rule:** A new number goes to **left** if it's $\leq$ the current median candidate (**left.top()**), otherwise to **right**.
- **Rebalance rule:** Ensure **left.size()** is either equal to **right.size()** or exactly 1 more. This guarantees:
 - Odd count $\rightarrow$ median = **left.top()**.
 - Even count $\rightarrow$ median = average of **left.top()** and **right.top()**.
- **Python:** **heapq** is min-heap only, so we store negatives in **left** to simulate max heap.

11. Complexity Analysis

Operation	Time	Space
addNum()	$O(\log n)$	$O(1)$
findMedian()	$O(1)$	$O(1)$
Total space	—	$O(n)$

12. Common Patterns

Pattern	Approach
Running median	Two heaps as described
Sliding window median	Two heaps + lazy deletion (or multiset)

Pattern	Approach
Find median of two sorted arrays	Binary search ($O(\log \min(n,m))$)

13. Common Mistakes

- **Forgetting to rebalance** — the size invariant must be checked after every insertion.
- **Using wrong comparator** — `left` must be max heap, `right` must be min heap.
- **Negation bug in Python** — forgetting to negate when pushing/popping from `left`.
- **Integer division** — in C++, `(left.top() + right.top()) / 2` does integer division. Use `2.0`.

14. Edge Cases

- **Single element:** Works — left has it, right is empty.
- **Two elements:** left gets one, right gets one (after rebalancing even size).
- **All equal elements:** They all go to `left` (since `num <= left.top()`), then rebalancing moves half to `right`.
- **Large number of elements:** Heap operations stay $O(\log n)$.

15. Variations

Variation	Change
Sliding window median	Add removal support via lazy deletion (or use <code>multiset</code>).
Find median of two sorted arrays	Binary search, $O(\log \min(n,m))$ — no heap needed.
Median of K sorted arrays	Min heap merges + tracking total count.

16. Related

- **Quickselect** — $O(n)$ to find median of static array.
- **Order statistic tree** — balanced BST with subtree sizes can find median in $O(\log n)$ and supports arbitrary deletions.

8. Top K Frequent Elements

1. Overview

Given an array of integers, find the K most frequent elements. This combines counting (hash map) with a heap to efficiently track the top K by frequency.

2. Intuition

Simple explanation: First, count how many times each element appears (using a hash map). Then, find the K elements with the highest counts — this is exactly the "K largest" problem, where the "value" is the frequency.

Why combine hash map + heap? The hash map gives us frequencies; the heap lets us find top K without sorting all N elements. If there are M unique elements, we need $O(M \log K)$ with heap vs $O(M \log M)$ with

sorting.

3. When to Use It

- "Top K frequent elements".
- "K most frequent words".
- "K most frequent numbers/characters".
- Any "top K by frequency" problem.

4. When Not to Use It

- K is close to M (unique elements): sorting is simpler and not much slower.
- Only one element needed (most frequent): just scan the hash map in $O(M)$.
- Array is small: sorting all elements by frequency is fine.

5. Core Concepts

Concept	Explanation
Frequency map	<code>unordered_map<element, frequency></code> .
Min heap of (freq, element)	Heap of size K, sorted by frequency. The root is the Kth most frequent so far.
Bucket sort alternative	Create an array of size N (max frequency) and put elements into buckets by frequency. $O(N)$ but may use more memory.

6. Step-by-Step Algorithm

1. Build frequency map: count occurrences of each element.
2. Create a min heap (size K) of pairs `(frequency, element)`.
3. For each `(element, freq)` in the map:
 - Push `(freq, element)` into the heap.
 - If heap size > K, pop the smallest frequency.
4. The heap now contains the K most frequent elements.
5. (Optional) Extract elements from heap for the result.

7. Dry Run

Array: [1, 1, 1, 2, 2, 3], K = 2

Step 1: Frequency map: {1: 3, 2: 2, 3: 1}

Step 2: Min heap of size 2

Entry (freq, val)	Heap	Size check
(3, 1)	[(3,1)]	OK
(2, 2)	[(2,2), (3,1)]	OK
(1, 3)	[(1,3), (3,1), (2,2)] → pop (1,3) → [(2,2), (3,1)]	>K, pop

Final heap: [(2,2), (3,1)] → elements {2, 1}

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

vector<int> topKFrequent(vector<int>& nums, int k) {
    // Step 1: Count frequencies
    unordered_map<int, int> freq;
    for (int x : nums) freq[x]++;

    // Step 2: Min heap of (frequency, value)
    using pii = pair<int, int>;
    priority_queue<pii, vector<pii>, greater<pii>> minHeap;

    for (auto& [val, count] : freq) {
        minHeap.push({count, val});
        if ((int)minHeap.size() > k)
            minHeap.pop();
    }

    // Step 3: Extract results
    vector<int> result;
    while (!minHeap.empty()) {
        result.push_back(minHeap.top().second);
        minHeap.pop();
    }
    return result;
}
```

9. Python Implementation

```
import heapq
from collections import Counter
from typing import List

def top_k_frequent(nums: List[int], k: int) -> List[int]:
    # Step 1: Count frequencies
    freq = Counter(nums)

    # Step 2: Min heap of size k
    min_heap = []
    for val, count in freq.items():
        heapq.heappush(min_heap, (count, val))
        if len(min_heap) > k:
            heapq.heappop(min_heap)

    # Step 3: Extract results
    return [val for _, val in min_heap]
```

```
# --- Alternative: bucket sort (O(N) time) ---
def top_k_frequent_bucket(nums: List[int], k: int) -> List[int]:
    freq = Counter(nums)
    # Buckets: index = frequency, value = list of elements with that frequency
    bucket = [[] for _ in range(len(nums) + 1)]
    for val, count in freq.items():
        bucket[count].append(val)

    result = []
    for count in range(len(bucket) - 1, 0, -1):
        for val in bucket[count]:
            result.append(val)
            if len(result) == k:
                return result
    return result

# --- Example ---
nums = [1, 1, 1, 2, 2, 3]
print(top_k_frequent(nums, 2)) # [1, 2] (or [2, 1])
```

10. Code Explanation

- **Step 1:** `unordered_map` / `Counter` counts frequencies in $O(N)$.
- **Step 2:** Min heap of size K stores `(frequency, value)`. The heap keeps the K largest frequencies by evicting the smallest when size exceeds K .
- **Step 3:** Extract values from the heap. Note that the heap does not preserve original order — we just need the elements.
- **Bucket sort alternative:** If N (array length) is not too large, we can bucket elements by frequency. This is $O(N)$ instead of $O(M \log K)$, where M is unique elements. But bucket size = $N+1$, which may be large.

11. Complexity Analysis

Approach	Time	Space
Heap	$O(N + M \log K)$	$O(N + K)$
Bucket sort	$O(N)$	$O(N)$
Sort entire map	$O(N + M \log M)$	$O(N)$

Where N = array length, M = unique elements, K = number to return.

12. Common Patterns

Pattern	Approach
Top K frequent elements	Hash map + min heap

Pattern	Approach
Top K frequent words	Same + lexicographic tiebreaker in comparator
K closest points to origin	Hash map + max heap (by distance)

13. Common Mistakes

- **Using max heap instead of min heap** for the top K — you'd keep the K largest in a max heap, but then the largest elements would never be evicted, and the heap could grow unbounded. A min heap of size K correctly evicts the smallest of the top K.
- **Forgetting to check if heap size > K** — without this, the heap stores all M unique elements.
- **Confusing frequency with value** in the pair order. For heap, frequency must be first so the heap sorts by it.

14. Edge Cases

- $K = 0$: return empty.
- $K \geq$ number of unique elements: return all unique elements.
- All elements same: frequency map has one entry; return it if $K \geq 1$.
- Negative numbers: works fine — frequencies are always non-negative.

15. Variations

Variation	Change
K most frequent words	String keys; if tie, use lexicographic order (smaller string first)
Top K frequent in stream	Use count-min sketch (approximate) or exact with heap
Kth most frequent	Same heap, just return root at end

16. Related

- **Sorting by frequency** — straightforward but $O(M \log M)$.
- **Quickselect on frequency array** — $O(M)$ average, but heap is simpler.

9. Custom Comparator

1. Overview

In many heap problems, the default ordering (by value or by tuple's first element) isn't enough. You need a **custom comparator** to define how elements in the priority queue should be ordered.

2. Intuition

Simple explanation: A comparator tells the heap: "Given two elements A and B, which one should be at the top?" In C++, the comparator returns **true** if the first argument should be placed **after** the second (i.e., has lower priority). In Python, we manipulate tuple elements or wrap objects to define ordering.

Analogy: You're sorting people by age, but if ages are equal, you want taller people first. You need a custom rule that combines multiple criteria.

3. When to Use It

- Priority depends on computed values (distance from origin, absolute difference, etc.).
- Multiple sorting criteria (primary key + tiebreaker).
- You're storing complex objects (structs, pairs, custom classes) and need a specific ordering.
- You need a min heap of pointers (C++).

4. When Not to Use It

- Simple integer ordering: default comparator is sufficient.
- You can encode the ordering by storing a different value (e.g., negate integers for max heap in Python).

5. Core Concepts

Concept	C++	Python
Default	<code>less<T></code> → max heap	<code>heapq</code> → min heap
Min heap	<code>greater<T></code>	Use negatives
Custom functor	<code>struct Compare { bool operator()(T a, T b) { ... } }</code>	Use tuple (priority, item) or define <code>__lt__</code>
Lambda	<code>auto cmp = [](T a, T b) { return ...; }</code>	Not directly usable with <code>heapq</code>
True = lower priority	Return <code>true</code> if <code>a</code> should be after <code>b</code>	N/A (<code>heapq</code> uses tuple comparison)

6. Step-by-Step Algorithm

Not applicable — this is about API usage patterns.

7. Dry Run

Not applicable — this is about API usage patterns.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// --- Struct with custom comparator ---
struct Point {
    int x, y;
    int dist() const { return x*x + y*y; }
};

struct ComparePoint {
```

```

// Min heap by distance from origin
bool operator()(const Point& a, const Point& b) {
    return a.dist() > b.dist(); // true → a has lower priority (placed after
b)
}
};

// --- Using lambda ---
auto cmp = [](int a, int b) {
    // Custom: prioritize even numbers over odd, then by value
    if ((a % 2) != (b % 2))
        return (a % 2) > (b % 2); // odd → lower priority
    return a > b; // larger → lower priority (min heap)
};
priority_queue<int, vector<int>, decltype(cmp)> customPQ(cmp);

// --- Using function pointer ---
bool myCompare(int a, int b) { return a > b; }
priority_queue<int, vector<int>, decltype(&myCompare)> fnPQ(myCompare);

// --- Priority queue of pairs with custom ordering ---
// Pair: (priority, value), sort by priority only
using pii = pair<int, int>;
struct ComparePair {
    bool operator()(const pii& a, const pii& b) {
        if (a.first != b.first)
            return a.first > b.first; // min heap by first
        return a.second > b.second; // tiebreaker: smaller second has higher
priority
    }
};
};

```

9. Python Implementation

```

import heapq
from typing import List, Tuple

# --- Use tuple (priority, value) – heapq compares tuples lexicographically ---
# Min heap by (priority, value)
h = []
heapq.heappush(h, (5, "task A"))
heapq.heappush(h, (1, "task B"))
heapq.heappush(h, (3, "task C"))

# --- For max heap with custom priorities: negate ---
max_h = []
heapq.heappush(max_h, (-5, "task A"))

# --- For complex objects: use a wrapper with __lt__ ---
class Task:
    def __init__(self, priority: int, name: str):

```

```

    self.priority = priority
    self.name = name

def __lt__(self, other):
    # Min heap by priority, then by name
    if self.priority != other.priority:
        return self.priority < other.priority
    return self.name < other.name

# heapq uses __lt__ for comparison
tasks = []
heapq.heappush(tasks, Task(5, "low"))
heapq.heappush(tasks, Task(1, "high"))
heapq.heappush(tasks, Task(3, "medium"))

while tasks:
    t = heapq.heappop(tasks)
    print(t.name) # high, medium, low

# --- For objects you can't modify: push (priority, index, object) ---
class ExternalObj:
    def __init__(self, val):
        self.val = val

objs = [ExternalObj(5), ExternalObj(1), ExternalObj(3)]
h = []
for i, obj in enumerate(objs):
    heapq.heappush(h, (obj.val, i, obj)) # tiebreaker = index

```

10. Code Explanation

C++ `priority_queue` comparator:

- The comparator `cmp(a, b)` must return `true` if `a` should have **lower priority** than `b` (i.e., `a` goes after `b` in the order of removal).
- For a **min heap** of integers, use `greater<int>` or `return a > b`.
- For a **max heap**, use `less<int>` (default) or `return a < b`.
- Custom struct: define `operator()` as a const member function.

Python `heapq` custom priority:

- Simplest: store `(priority, item)` tuples. `heapq` compares tuples element by element.
- For max heap: negate the priority value.
- For custom objects: define `__lt__(self, other)` — `heapq` uses only `<` for comparison.
- For objects you can't modify: use `(priority, tiebreaker, object)` tuples.

11. Complexity Analysis

No additional complexity — the comparator runs in $O(1)$ per comparison, and heap ops remain $O(\log n)$.

12–16. (Same patterns as earlier sections)

10. Heap Sort

1. Overview

Heap sort is a comparison-based sorting algorithm that uses a binary heap. It has $O(n \log n)$ worst-case time complexity and sorts in-place. It is not stable (equal elements may change relative order).

2. Intuition

Simple explanation: Build a max heap from the array, then repeatedly extract the maximum element (root) and place it at the end of the array. The array gets sorted from right to left.

Analogy: You have a pile of differently sized rocks. You build a pyramid with the largest rock on top. Then you keep removing the top rock and placing it aside. The rocks come out in decreasing order.

Why it works: The heap property guarantees the largest remaining element is always at the root. Extracting it gives the next element in sorted order.

3. When to Use It

- You need **$O(n \log n)$ worst-case** time (unlike quicksort's $O(n^2)$ worst case).
- You need **in-place** sorting with $O(1)$ extra space.
- Embedded systems or memory-constrained environments.

4. When Not to Use It

- You need **stable** sorting — use merge sort.
- You need the fastest average-case sort — use **introsort** (C++ `std::sort`) or **quicksort**.
- The array is almost sorted — use **insertion sort** ($O(n)$ on nearly sorted data).
- You need **external sorting** — use merge sort.
- Data doesn't support random access — heap sort requires array indexing.

5. Core Concepts

Concept	Explanation
Build max heap	Convert array into a max heap using Floyd's $O(n)$ method.
Extract max	Swap root with last element, reduce heap size, bubble down new root.
In-place	The sorted portion grows from the right, the heap shrinks from the right.

6. Step-by-Step Algorithm

1. **Build max heap** from the array ($O(n)$).
2. For $i = n-1$ down to 1 :
 - Swap `arr[0]` (root) with `arr[i]` (last element of heap).
 - Reduce heap size by 1.
 - Call `heapify(arr, i, 0)` to restore max heap on the reduced heap.

3. The array is now sorted in ascending order.

7. Dry Run

Array: [4, 10, 3, 5, 1]

Step 1: Build max heap

Initial:

```

      4
     / \
    10  3
   /  \
  5    1

```

After heapify at index 1 (10 → 10 > 5 and 1, no change):

```

      4
     / \
    10  3
   /  \
  5    1

```

After heapify at index 0 (4 → swap with 10):

```

     10
    /  \
   4    3
  /  \
 5    1

```

After heapify at index 0 (4 → swap with 5):

```

     10
    /  \
   5    3
  /  \
 4    1

```

Built heap: [10, 5, 3, 4, 1]

Step 2: Extract repeatedly

i=4: Swap 10,1 → [1, 5, 3, 4, 10]. Heapify(0) on size 4:
1→5→4 → [5, 4, 3, 1, 10]

i=3: Swap 5,1 → [1, 4, 3, 5, 10]. Heapify(0) on size 3:
1→4 → [4, 1, 3, 5, 10]

i=2: Swap 4,3 → [3, 1, 4, 5, 10]. Heapify(0) on size 2:
3→1 → [3, 1, 4, 5, 10] (wait, 3 > 1, no swap)

i=1: Swap 3,1 → [1, 3, 4, 5, 10]. Heapify(0) on size 1: done.

Final sorted: [1, 3, 4, 5, 10]

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

void heapify(vector<int>& arr, int n, int i) {
    int largest = i;
    int l = 2 * i + 1, r = 2 * i + 2;

    if (l < n && arr[l] > arr[largest]) largest = l;
    if (r < n && arr[r] > arr[largest]) largest = r;

    if (largest != i) {
        swap(arr[i], arr[largest]);
        heapify(arr, n, largest);
    }
}

void heapSort(vector<int>& arr) {
    int n = arr.size();

    // Build max heap
    for (int i = n / 2 - 1; i >= 0; --i)
        heapify(arr, n, i);

    // Extract elements one by one
    for (int i = n - 1; i > 0; --i) {
        swap(arr[0], arr[i]); // Move current root to end
        heapify(arr, i, 0);   // Call heapify on reduced heap
    }
}

// --- Example ---
int main() {
    vector<int> arr = {4, 10, 3, 5, 1};
    heapSort(arr);
    for (int x : arr) cout << x << " "; // 1 3 4 5 10
    return 0;
}
```

9. Python Implementation

```
def heapify(arr, n, i):
    """Max heapify (bubble down) for heap sort"""
    largest = i
    l, r = 2 * i + 1, 2 * i + 2
```

```
    if l < n and arr[l] > arr[largest]:
        largest = l
    if r < n and arr[r] > arr[largest]:
        largest = r

    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest)

def heap_sort(arr):
    n = len(arr)

    # Build max heap
    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)

    # Extract elements
    for i in range(n - 1, 0, -1):
        arr[0], arr[i] = arr[i], arr[0] # Move max to end
        heapify(arr, i, 0)             # Heapify on reduced heap

    return arr

# --- Example ---
arr = [4, 10, 3, 5, 1]
print(heap_sort(arr)) # [1, 3, 4, 5, 10]
```

10. Code Explanation

- **Build phase:** Floyd's $O(n)$ method. Start from last parent and heapify each node.
- **Sort phase:** Swap root (maximum) with the last element, then heapify the reduced heap (excluding the sorted suffix). This shrinks the heap from the right while the sorted portion grows from the right.
- **Result:** Ascending order. For descending order, use a min heap.

11. Complexity Analysis

Phase	Time	Space
Build heap	$O(n)$	$O(1)$ extra
Sort ($n-1$ extractions)	$O(n \log n)$	$O(1)$ extra
Total	$O(n \log n)$	$O(1)$

12. Common Patterns

Pattern	Approach
Sort in-place with $O(1)$ space	Heap sort
Partial sort (smallest K)	Build min heap, pop K times

13. Common Mistakes

- **Using 1-based indexing** for children ($2i$ / $2i+1$) with 0-based array — use $2i+1$ / $2i+2$.
- **Forgetting to pass correct heap size** to **heapify** — after extraction, the heap size is **i**, not **n**.
- **Not building heap first** — sorting directly on an unheaped array gives wrong results.

14. Edge Cases

- **Empty array:** No sorting needed.
- **Single element:** Already sorted.
- **All equal elements:** Heapify doesn't swap, sort produces the same array.
- **Already sorted:** Still $O(n \log n)$ — no early termination.
- **Reverse sorted:** Also $O(n \log n)$.

15. Variations

Variation	Change
Descending order	Use min heap instead of max heap
Partial heap sort	Only extract K elements (for K largest/smallest)
Introspective sort (introsort)	Combines quicksort, heap sort, and insertion sort; heap sort used as fallback to avoid $O(n^2)$ worst case

16. Related

- **Quicksort** — $O(n \log n)$ average, $O(n^2)$ worst, but faster in practice.
- **Merge sort** — $O(n \log n)$ stable, needs $O(n)$ extra space.
- **C++ `std::sort`** — introsort; use this instead of heap sort unless you specifically need in-place sorting with $O(1)$ extra space.

11. Two Heaps Technique

1. Overview

The **two heaps technique** uses one max heap and one min heap to efficiently maintain a dynamic set of elements where you need access to both the smallest of the larger half and the largest of the smaller half.

2. Intuition

Simple explanation: Split the data into two halves at the median. A max heap stores the left (smaller) half, and a min heap stores the right (larger) half. The median or any quantile is always one of the tops.

Why it works: The max heap's root is the largest element in the left half (the "middle" candidate). The min heap's root is the smallest element in the right half. Together, they bracket the middle of the data.

3. When to Use It

- **Running median** (classic use).
- **Sliding window median**.
- Tracking the **middle element(s)** of a dynamic stream.
- **Dynamic percentile** tracking.
- Problems where you need to **balance two halves** of data.

4. When Not to Use It

- You only need the minimum or maximum — a single heap is enough.
- You need arbitrary percentiles — use an order statistic tree.
- Data is static and you only query once — sort and index.

5. Core Concepts

Concept	Explanation
Max heap (left)	Holds the smaller half. Root = largest of the smaller half.
Min heap (right)	Holds the larger half. Root = smallest of the larger half.
Size invariant	Left size = right size OR left = right + 1.
Median from two heaps	Odd: <code>left.top()</code> . Even: $(\text{left.top()} + \text{right.top()}) / 2$.

6. Step-by-Step Algorithm

(Already covered in Running Median section.)

7. Dry Run

(Already covered in Running Median section.)

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

template<typename T>
class TwoHeaps {
private:
    priority_queue<T> left;                // max heap
    priority_queue<T, vector<T>, greater<T>> right; // min heap

public:
    void add(T num) {
        if (left.empty() || num <= left.top())
```

```

        left.push(num);
    else
        right.push(num);

    // Rebalance
    if (left.size() > right.size() + 1) {
        right.push(left.top());
        left.pop();
    } else if (right.size() > left.size()) {
        left.push(right.top());
        right.pop();
    }
}

double getMedian() {
    if (left.size() > right.size())
        return (double)left.top();
    return (left.top() + right.top()) / 2.0;
}

void remove(T num) {
    // Lazy deletion uses a third data structure (e.g., unordered_map)
    // Not supported directly without tracking
}

int size() { return (int)(left.size() + right.size()); }
};

```

9. Python Implementation

```

import heapq

class TwoHeaps:
    def __init__(self):
        self.left = [] # max heap (store negatives)
        self.right = [] # min heap

    def add_num(self, num):
        if not self.left or num <= -self.left[0]:
            heapq.heappush(self.left, -num)
        else:
            heapq.heappush(self.right, num)

        # Rebalance
        if len(self.left) > len(self.right) + 1:
            heapq.heappush(self.right, -heapq.heappop(self.left))
        elif len(self.right) > len(self.left):
            heapq.heappush(self.left, -heapq.heappop(self.right))

    def find_median(self):
        if len(self.left) > len(self.right):

```

```
return -self.left[0]
return (-self.left[0] + self.right[0]) / 2.0
```

10. Code Explanation

(Already covered in Running Median section.)

11. Complexity Analysis

Operation	Time
<code>add(T)</code>	$O(\log n)$
<code>getMedian()</code>	$O(1)$

12. Common Patterns

Pattern	Approach
Find median from data stream	Two heaps
Sliding window median	Two heaps + lazy deletion (using <code>unordered_map</code> to track removals)
IPO (Maximum Capital)	Two heaps: one for affordable projects (max by profit), one for all projects (min by capital)
Find right interval	Two heaps on intervals

13. Common Mistakes

- **Wrong rebalancing condition** — `left.size() > right.size() + 1` is the correct check for "left too big".
- **Forgetting tiebreaker** — in Python, if two tuples have the same priority, `heapq` uses the next element; ensure a tiebreaker exists to avoid comparing objects that don't support `<`.
- **Not handling the case where left is empty** on `remove()`.

14. Edge Cases

- **Single element:** Left heap has it, right is empty. Median = `left.top()`.
- **Two elements:** Left has one, right has one (or left has both before rebalance). Works correctly.
- **Duplicates:** Handled naturally; both heaps accept equal values.

15. Variations

Variation	Change
Sliding window median	Add lazy deletion for removals
Dynamic quantile	Adjust the size ratio (e.g., keep 25th percentile by maintaining 1:3 ratio)

Variation	Change
Three heaps	For tracking two pivot points (e.g., 25th and 75th percentile)

16. Related

- **Segment tree / Fenwick tree** — can find arbitrary order statistics, more powerful but more complex.
- **Balanced BST with subtree sizes** — can also find median + support deletions.

12. Dijkstra Using Priority Queue

1. Overview

Dijkstra's algorithm finds the shortest paths from a source node to all other nodes in a weighted graph with **non-negative** edge weights. A priority queue (min heap) is used to efficiently select the next node with the smallest tentative distance.

2. Intuition

Simple explanation: Start from the source node with distance 0. Keep a "tentative distance" for every node. At each step, pick the unvisited node with the smallest tentative distance, mark it as visited (its distance is now final), and update its neighbors' distances.

Why a priority queue? Without a PQ, you'd scan all unvisited nodes to find the smallest distance — $O(V^2)$. A PQ reduces edge relaxation to $O(\log V)$ per edge, giving $O(E \log V)$ overall.

Analogy: Imagine you're at a central train station. You want to know the shortest travel time to every other station. You maintain a list of "best known times" for each station. At each step, you go to the station you can reach fastest among those you haven't visited yet. From there, you check if going through it gives a shorter route to other stations.

3. When to Use It

- **Shortest path** in a weighted graph with **non-negative** edges.
- **Single source shortest path** (SSSP).
- Variants: shortest path in grid, network delay time, cheapest flights with K stops.

4. When Not to Use It

- **Negative edges** → use Bellman-Ford or SPFA.
- **Unweighted graph** → BFS is simpler and $O(V+E)$.
- **Need all-pairs shortest path** → Floyd-Warshall or run Dijkstra from each node (if sparse).
- **Graph is very dense** ($E \approx V^2$) → $O(V^2)$ array-based Dijkstra may be faster than $O(E \log V)$ heap-based.

5. Core Concepts

Concept	Explanation
Distance array	$\text{dist}[v]$ = shortest known distance from source to v .
Visited set	Nodes whose shortest distance has been finalized.
Relaxation	If $\text{dist}[u] + w(u,v) < \text{dist}[v]$, update $\text{dist}[v]$.
PQ of (distance, node)	Min heap ordered by distance. Always gives the node with smallest tentative distance.

6. Step-by-Step Algorithm

1. Initialize $\text{dist}[\text{source}] = 0$, $\text{dist}[\text{others}] = \text{INF}$.
2. Push $(0, \text{source})$ into min heap.
3. While heap is not empty:
 - Pop (d, u) . If $d > \text{dist}[u]$, skip (outdated entry).
 - For each neighbor v of u with edge weight w :
 - If $\text{dist}[u] + w < \text{dist}[v]$:
 - Update $\text{dist}[v] = \text{dist}[u] + w$.
 - Push $(\text{dist}[v], v)$ into heap.
4. dist now contains shortest distances from source.

7. Dry Run

Graph:

```

0 --4-- 1 --8-- 3
|       |       |
2       1       4
|       |       |
2 --5-- 4 --9-- 5

```

Source = 0.

(Using a cleaner example):

```

Nodes: 0, 1, 2, 3, 4
Edges:
0→1 (4), 0→2 (2)
1→2 (1), 1→3 (5)
2→3 (8), 2→4 (10)
3→4 (2)

Source = 0

```

Step	Heap (dist, node)	Pop	dist[0]	dist[1]	dist[2]	dist[3]	dist[4]
Init	(0,0)	—	0	∞	∞	∞	∞
1	—	(0,0)	0	4	2	∞	∞
2	(2,2), (4,1)	(2,2)		3 (via 2)	2	10	12
3	(3,1), (4,1), (10,3), (12,4)	(3,1)		3		8 (via 1)	
4	(4,1), (8,3), (10,3), (12,4)	(4,1)*					
5	(8,3), (10,3), (12,4)	(8,3)				8	10 (via 3)
6	(10,3), (10,4), (12,4)	(10,3)*					
7	(10,4), (12,4)	(10,4)					10

- = outdated entry (distance doesn't match current dist), skipped.

Final distances: [0, 3, 2, 8, 10]

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

using pii = pair<int, int>;
const int INF = 1e9;

vector<int> dijkstra(int source, int V, vector<vector<pii>>& adj) {
    // adj[u] = list of {v, weight}
    vector<int> dist(V, INF);
    dist[source] = 0;

    // Min heap of {distance, node}
    priority_queue<pii, vector<pii>, greater<pii>> pq;
    pq.push({0, source});

    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();

        // Skip if outdated (not the current best distance)
        if (d != dist[u]) continue;

        for (auto& [v, w] : adj[u]) {
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
        }
    }

    return dist;
}
```

```

}

// --- Example ---
int main() {
    int V = 5;
    vector<vector<pii>> adj(V);

    adj[0].push_back({1, 4});
    adj[0].push_back({2, 2});
    adj[1].push_back({2, 1});
    adj[1].push_back({3, 5});
    adj[2].push_back({3, 8});
    adj[2].push_back({4, 10});
    adj[3].push_back({4, 2});

    vector<int> dist = dijkstra(0, V, adj);
    for (int i = 0; i < V; ++i)
        cout << "dist[" << i << "] = " << dist[i] << "\n";
    return 0;
}

```

9. Python Implementation

```

import heapq
from typing import List, Tuple

INF = 10**9

def dijkstra(source: int, V: int, adj: List[List[Tuple[int, int]]]) -> List[int]:
    """adj[u] = list of (v, weight)"""
    dist = [INF] * V
    dist[source] = 0

    # Min heap of (distance, node)
    pq = [(0, source)]

    while pq:
        d, u = heapq.heappop(pq)

        # Skip outdated entries
        if d != dist[u]:
            continue

        for v, w in adj[u]:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                heapq.heappush(pq, (dist[v], v))

    return dist

```

```
# --- Example ---
V = 5
adj = [[] for _ in range(V)]
adj[0] = [(1, 4), (2, 2)]
adj[1] = [(2, 1), (3, 5)]
adj[2] = [(3, 8), (4, 10)]
adj[3] = [(4, 2)]

dist = dijkstra(0, V, adj)
for i, d in enumerate(dist):
    print(f"dist[{i}] = {d}")
```

10. Code Explanation

- **dist array:** INF initially. `dist[source] = 0`.
- **PQ stores (distance, node):** Always pop the node with smallest known distance.
- **Outdated entry skip:** The same node may be pushed multiple times with different distances. We only process the entry where `d == dist[u]`. This is called **lazy deletion**.
- **Relaxation:** For each neighbor `v`, if the path through `u` is shorter, update `dist[v]` and push into PQ.

11. Complexity Analysis

Variant	Time	Space
PQ-based	$O((V+E) \log V) = O(E \log V)$	$O(V + E)$
Array-based (dense graph)	$O(V^2)$	$O(V^2)$

12. Common Patterns

Pattern	Approach
Shortest path in grid	Each cell is a node; 4-directional moves with weight, often 1
Network delay time	Dijkstra from source, find max distance to any reachable node
Cheapest flights with K stops	Bellman-Ford (if K is small) or Dijkstra with state (node, stops)
Path with minimum effort	Dijkstra where weight is max height difference on path (variant)
Number of shortest paths	Dijkstra + counting array (increment on equality)

13. Common Mistakes

- **Not skipping outdated entries** → processing stale distances, possible exponential blowup or wrong answers.
- **Using PQ with visited array** → updating visited on push instead of pop may miss shorter paths discovered later.
- **Forgetting directed vs undirected** → directed graphs add edges in one direction only.
- **INF too small** → sum of edge weights may overflow.
- **Not handling disconnected nodes** → `dist[v]` stays INF.

14. Edge Cases

- **Source equals target:** $\text{dist} = 0$.
- **Disconnected graph:** $\text{dist}[\text{unreachable}] = \text{INF}$.
- **Single node:** $\text{dist} = [0]$.
- **Graph with zero-weight edges:** Works fine (PQ will process in order).
- **Multiple edges between same nodes:** Handle by selecting the minimum weight during input or letting relaxation handle it.

15. Variations

Variation	Change
A*	Add heuristic to guide search; PQ ordered by dist + heuristic .
Dial's algorithm	Use bucket queue for graphs with small integer weights ($O(V+E+w)$).
Dijkstra with potentials	For Johnson's algorithm (all-pairs shortest paths on sparse graphs).
Bidirectional Dijkstra	Run from both source and target; halts when frontiers meet.

16. Related

- **BFS** — shortest path on unweighted graphs ($O(V+E)$).
- **Bellman-Ford** — handles negative edges ($O(VE)$).
- **Floyd-Warshall** — all-pairs shortest paths ($O(V^3)$).
- **Prim's MST** — same structure but minimizes total weight of MST, not path from source.

13. Fibonacci Heap Idea

1. Overview

A **Fibonacci heap** is a heap data structure with excellent amortized time bounds: $O(1)$ for insert, merge, and decrease-key, and $O(\log n)$ for extract-min. It is primarily of **theoretical interest** and used in practice only when decrease-key operations are very frequent.

2. Intuition

Simple explanation: A Fibonacci heap is a collection of heap-ordered trees. Unlike binary heaps, it doesn't maintain a strict shape — it's "lazy." It defers most of the work (consolidation, cutting) to extract-min, which is where the hard amortized work happens.

Why "Fibonacci"? The trees in a Fibonacci heap obey the Fibonacci numbers — a tree of degree k has at least F_{k+2} nodes. This ensures the height is $O(\log n)$.

Analogy: Imagine a lazy student who doesn't clean their desk until absolutely necessary. When inserting new books, they just toss them onto the pile ($O(1)$). When decreasing priority (marking a book as more important), they just move it to the top. Only when asked to find the most important book (extract-min) do they organize everything — but they organize cleverly, keeping it logarithmic overall.

3. When to Use It

- **Theoretically** optimal Dijkstra / Prim ($O(V \log V + E)$ instead of $O((V+E) \log V)$).
- Algorithms requiring many decrease-key operations.
- Understanding amortized analysis.

4. When Not to Use It

- **In practice**, binary heaps are faster for nearly all cases due to cache behavior and simplicity.
- **Pairing heaps** are simpler and often faster.
- **Competitive programming**: STL's `priority_queue` is sufficient. Fibonacci heaps are complex to implement and rarely needed.

5. Core Concepts

Concept	Explanation
Lazy insertion	New nodes are added to a root list without consolidation.
Consolidation	During extract-min, combine trees of the same degree until each degree appears at most once.
Marking	Nodes are "marked" when they lose a child. If a marked node loses another child, it is cut from its parent and moved to the root list (cascading cut).
Amortized analysis	The expensive operations in extract-min are paid for by the cheap operations that preceded them.

6. Step-by-Step Algorithm (Basic)

Insert: Add node to root list. Update min pointer if needed. $O(1)$.

Extract-min: Remove min node, add its children to root list. Consolidate trees of same degree. $O(\log n)$ amortized.

Decrease-key: Decrease key. If heap property violated, cut the node and move it to root list. If parent was marked, cut parent too (cascading). $O(1)$ amortized.

Merge: Concatenate root lists. $O(1)$.

7. Dry Run

(Too complex for a table; Fibonacci heap dry runs are typically done on paper with tree diagrams.)

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Simplified Fibonacci heap – for study only, not production use.
```

```

struct FibNode {
    int key;
    FibNode *parent, *child, *left, *right;
    int degree;
    bool marked;

    FibNode(int k) : key(k), parent(nullptr), child(nullptr),
                    left(this), right(this), degree(0), marked(false) {}
};

class FibHeap {
private:
    FibNode* minNode;
    int n;

    void cut(FibNode* x, FibNode* y) {
        // Remove x from child list of y, add to root list
        x->left->right = x->right;
        x->right->left = x->left;
        y->degree--;
        // Add to root list
        x->parent = nullptr;
        x->left = minNode;
        x->right = minNode->right;
        minNode->right->left = x;
        minNode->right = x;
        x->marked = false;
    }

    void cascadingCut(FibNode* y) {
        FibNode* z = y->parent;
        if (z) {
            if (!y->marked)
                y->marked = true;
            else {
                cut(y, z);
                cascadingCut(z);
            }
        }
    }

    void link(FibNode* y, FibNode* x) {
        // Make y a child of x
        y->left->right = y->right;
        y->right->left = y->left;
        y->parent = x;
        if (!x->child) {
            x->child = y;
            y->left = y->right = y;
        } else {
            y->left = x->child;
            y->right = x->child->right;
            x->child->right->left = y;
            x->child->right = y;
        }
    }
};

```



```

    }
    x->degree++;
    y->marked = false;
}

void consolidate() {
    int maxDegree = (int)(log2(n)) + 1;
    vector<FibNode*> degreeTable(maxDegree, nullptr);

    // Collect root nodes
    vector<FibNode*> roots;
    if (minNode) {
        FibNode* cur = minNode;
        do {
            roots.push_back(cur);
            cur = cur->right;
        } while (cur != minNode);
    }

    for (FibNode* w : roots) {
        FibNode* x = w;
        int d = x->degree;
        while (d < maxDegree && degreeTable[d]) {
            FibNode* y = degreeTable[d];
            if (x->key > y->key) swap(x, y);
            link(y, x);
            degreeTable[d] = nullptr;
            d++;
        }
        degreeTable[d] = x;
    }

    minNode = nullptr;
    for (FibNode* node : degreeTable) {
        if (node) {
            if (!minNode) {
                minNode = node;
                minNode->left = minNode->right = minNode;
            } else {
                node->left = minNode;
                node->right = minNode->right;
                minNode->right->left = node;
                minNode->right = node;
                if (node->key < minNode->key)
                    minNode = node;
            }
        }
    }
}

public:
    FibHeap() : minNode(nullptr), n(0) {}

    void push(int key) {

```

```
FibNode* node = new FibNode(key);
if (!minNode) {
    minNode = node;
} else {
    node->left = minNode;
    node->right = minNode->right;
    minNode->right->left = node;
    minNode->right = node;
    if (key < minNode->key)
        minNode = node;
}
n++;
}

int top() {
    if (!minNode) throw out_of_range("Empty heap");
    return minNode->key;
}

int pop() {
    if (!minNode) throw out_of_range("Empty heap");
    FibNode* oldMin = minNode;

    // Move children to root list
    if (oldMin->child) {
        FibNode* child = oldMin->child;
        do {
            child->parent = nullptr;
            child = child->right;
        } while (child != oldMin->child);

        // Merge child list into root list
        FibNode* firstChild = oldMin->child;
        FibNode* lastChild = firstChild->left;

        firstChild->left = oldMin->left;
        oldMin->left->right = firstChild;
        lastChild->right = oldMin->right;
        oldMin->right->left = lastChild;
    }

    // Remove min from root list
    oldMin->left->right = oldMin->right;
    oldMin->right->left = oldMin->left;

    if (oldMin == oldMin->right) {
        minNode = nullptr;
    } else {
        minNode = oldMin->right;
        consolidate();
    }

    int key = oldMin->key;
    delete oldMin;
}
```

```

        n--;
        return key;
    }

    bool empty() { return n == 0; }
    int size() { return n; }
};

```

Note: This is a simplified implementation for understanding. Production code would be more robust. In contests/placements, you almost never need to implement Fib heap — use STL `priority_queue`.

9. Python Implementation

(Not practical to implement in Python for CP/placements. Use `heapq`.)

10. Code Explanation

- **Root list:** Doubly linked circular list of all root nodes.
- **Consolidation:** After extract-min, trees of the same degree are linked together. Uses an array indexed by degree.
- **Cascading cut:** During decrease-key, if a node loses a second child, it is cut and moved to root list. This ensures the tree structure remains shallow.
- **Linking:** In consolidation, the node with larger key becomes a child of the node with smaller key.

11. Complexity Analysis

Operation	Amortized Time
<code>push()</code>	$O(1)$
<code>pop()</code>	$O(\log n)$
<code>decrease-key()</code>	$O(1)$
<code>merge()</code>	$O(1)$
<code>top()</code>	$O(1)$

12–14. (Theoretical; practical interview relevance is low.)

15. Variations

- **Fibonacci heap vs Pairing heap vs Binomial heap** — see next section.
- **Relaxed heap** — Another variation with similar bounds.

16. Related

- **Binomial heap** — $O(\log n)$ for all operations, supports fast merge. Simpler than Fibonacci.
- **Pairing heap** — practical alternative with good average performance.
- **Binary heap** — simplest, best for most use cases.

14. Pairing Heap Idea

1. Overview

A **pairing heap** is a heap data structure that is simpler than Fibonacci heaps but has good practical performance. It is implemented as a multi-way tree (each node can have many children). It achieves $O(\log n)$ amortized time for extract-min and $O(1)$ for insert and merge.

2. Intuition

Simple explanation: Each node has a value and a list of children (sub-heaps). The root has the minimum value. When you need to extract the minimum, you merge all its children together — two at a time — using a technique called **pairing** (hence the name).

Analogy: Think of a tournament bracket. The winner (minimum) is at the top. When the winner leaves, the remaining players need to compete again. Pairing heaps do this efficiently by pairing up the children and merging in a clever order.

Why pairing? The "pairing" merge — merging children in pairs, then merging the results in sequence — gives the best amortized bounds. It's like a tournament where each match combines two trees into one.

3. When to Use It

- When you need a **decrease-key** operation and the Fibonacci heap is too complex.
- When you want good **practical performance** with simpler code than Fibonacci.
- In **functional programming** (pairing heaps are easy to implement in purely functional languages).

4. When Not to Use It

- For CP/placements — STL `priority_queue` is simpler and fast enough.
- When you need **worst-case guarantees** — pairing heap has $O(n)$ worst case for some operations (though amortized is good).
- When the dataset is small — binary heap is simpler.

5. Core Concepts

Concept	Explanation
Multi-way tree	Each node has a value and a linked list of child sub-heaps.
Pairing merge	Combining two heaps by making the larger root a child of the smaller root.
Two-pass merging	After extract-min, merge children in pairs from left to right, then merge results from right to left.
Decrease-key	Cut the node from its parent, decrease the key, then merge it back into the root.

6. Step-by-Step Algorithm

Insert: Merge the new node (as a single-element heap) with the existing heap. $O(1)$.

Merge two heaps: Compare roots. The smaller root becomes the parent of the larger root. $O(1)$.

Extract-min: Remove root. Merge its children using two-pass pairing: first pass pairs adjacent children; second pass merges the resulting heaps in sequence. $O(\log n)$ amortized.

7. Dry Run

(Visual tree manipulation is best shown with diagrams. In text: extract-min from a root with 4 children merges (child1, child2) $\rightarrow$ heapA, (child3, child4) $\rightarrow$ heapB, then merges heapA with heapB.)

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

struct PairingNode {
    int key;
    PairingNode *child, *sibling;

    PairingNode(int k) : key(k), child(nullptr), sibling(nullptr) {}
};

// Merge two pairing heaps (compare roots)
PairingNode* merge(PairingNode* a, PairingNode* b) {
    if (!a) return b;
    if (!b) return a;
    if (a->key > b->key) swap(a, b);
    // Make b a child of a
    b->sibling = a->child;
    a->child = b;
    return a;
}

// Two-pass pairing merge
PairingNode* mergePairs(PairingNode* head) {
    if (!head || !head->sibling) return head;

    // First pass: pair adjacent siblings
    PairingNode* a = head;
    PairingNode* b = head->sibling;
    PairingNode* rest = b->sibling;

    a->sibling = b->sibling = nullptr;
    PairingNode* paired = merge(a, b);

    // Recursively merge the rest
    PairingNode* remaining = mergePairs(rest);

    // Second pass: merge the pair with the remaining
    return merge(paired, remaining);
}
```

```

class PairingHeap {
private:
    PairingNode* root;

public:
    PairingHeap() : root(nullptr) {}

    void push(int key) {
        PairingNode* node = new PairingNode(key);
        root = merge(root, node);
    }

    int top() {
        if (!root) throw out_of_range("Empty heap");
        return root->key;
    }

    int pop() {
        if (!root) throw out_of_range("Empty heap");
        int minKey = root->key;
        PairingNode* oldRoot = root;
        root = mergePairs(root->child);
        delete oldRoot;
        return minKey;
    }

    bool empty() { return !root; }

    void merge(PairingHeap& other) {
        root = merge(root, other.root);
        other.root = nullptr;
    }
};

// --- Example ---
int main() {
    PairingHeap ph;
    ph.push(5); ph.push(2); ph.push(8); ph.push(1); ph.push(3);
    while (!ph.empty()) {
        cout << ph.pop() << " "; // 1 2 3 5 8
    }
    return 0;
}

```

9. Python Implementation

```

class PairingNode:
    def __init__(self, key):
        self.key = key
        self.child = None
        self.sibling = None

```

```
def merge(a, b):
    """Merge two pairing heaps"""
    if a is None:
        return b
    if b is None:
        return a
    if a.key > b.key:
        a, b = b, a
    # b becomes a child of a
    b.sibling = a.child
    a.child = b
    return a

def merge_pairs(head):
    """Two-pass pairing merge"""
    if head is None or head.sibling is None:
        return head

    # First pass: pair adjacent siblings
    a = head
    b = head.sibling
    rest = b.sibling

    a.sibling = None
    b.sibling = None
    paired = merge(a, b)

    # Recursively process rest, then merge
    return merge(paired, merge_pairs(rest))

class PairingHeap:
    def __init__(self):
        self.root = None

    def push(self, key):
        node = PairingNode(key)
        self.root = merge(self.root, node)

    def top(self):
        if self.root is None:
            raise IndexError("Empty heap")
        return self.root.key

    def pop(self):
        if self.root is None:
            raise IndexError("Empty heap")
        min_key = self.root.key
        self.root = merge_pairs(self.root.child)
        return min_key
```

```
def __bool__(self):
    return self.root is not None

# --- Example ---
ph = PairingHeap()
for v in [5, 2, 8, 1, 3]:
    ph.push(v)
while ph:
    print(ph.pop(), end=" ") # 1 2 3 5 8
```

10. Code Explanation

- **Merge:** The smaller root becomes the parent of the larger root's entire tree. This is $O(1)$.
- **Two-pass pairing:** After extract-min, we need to merge all child sub-heaps. The first pass pairs them up (adjacent siblings). The second pass merges these pairs in sequence. This specific order gives the best amortized bound.
- **Decrease-key** (not shown): Cut the node, update its key, merge it back into the root. Needs a parent pointer.

11. Complexity Analysis

Operation	Amortized Time	Worst Case
<code>push()</code>	$O(1)$	$O(1)$
<code>pop()</code>	$O(\log n)$	$O(n)$
<code>merge()</code>	$O(1)$	$O(1)$
<code>decrease-key()</code>	$O(\log n)$ amortized	$O(n)$

12. When to Choose Pairing Heap vs Others

Heap Type	Best For
Binary heap	General use, CP, interviews — simplest to implement and use
Pairing heap	When you need decrease-key with simple code
Fibonacci heap	Theoretical optimal bounds; rarely needed in practice
Binomial heap	When you need fast merge and predictable $O(\log n)$ worst case

13–14. (See general Heap section for edge cases and mistakes.)

15. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
Kth Largest Element in an Array	LeetCode 215	Min heap of size K	Easy
Last Stone Weight	LeetCode 1046	Max heap	Easy

Medium

Problem	Platform	Pattern	Difficulty
Top K Frequent Elements	LeetCode 347	Hash map + min heap	Medium
Find Median from Data Stream	LeetCode 295	Two heaps	Medium
K Closest Points to Origin	LeetCode 973	Max heap of size K (or min heap)	Medium
Merge k Sorted Lists	LeetCode 23	Min heap + K-way merge	Medium
Task Scheduler	LeetCode 621	Max heap by frequency + cooldown	Medium

Hard

Problem	Platform	Pattern	Difficulty
Sliding Window Median	LeetCode 480	Two heaps + lazy deletion	Hard
IPO	LeetCode 502	Two heaps (capital + profit)	Hard
Minimum Cost to Hire K Workers	LeetCode 857	Max heap + sorting by wage ratio	Hard
Trapping Rain Water II	LeetCode 407	Min heap + BFS (3D water)	Hard
Smallest Range Covering Elements from K Lists	LeetCode 632	Min heap + sliding window	Hard

16. Interview Explanation

If asked "Explain heap / priority queue" in an interview:

"A **heap** is a complete binary tree where every parent is $\leq$ (min heap) or $\geq$ (max heap) its children. It's typically stored in an array: for node at index i , children are at $2i+1$ and $2i+2$, parent at $(i-1)/2$.

The key operations are:

- **Push** (insert): add at end, bubble up — $O(\log n)$.
- **Pop** (extract min/max): swap root with last, bubble down — $O(\log n)$.
- **Top** (peek): return root — $O(1)$.

A **priority queue** is the abstract data type that a heap usually implements. In C++, `priority_queue` is a max heap by default; use `greater<T>` for min heap. In Python, `heapq` is a min heap.

Key applications:

- **Kth largest/smallest:** maintain a heap of size K.
- **Running median:** two heaps — one max, one min.
- **Dijkstra's algorithm:** PQ gives us the next node with smallest tentative distance.
- **Merge K sorted lists:** push first element of each list, repeatedly pop and push the next.
- **Heap sort:** build max heap, repeatedly extract — $O(n \log n)$ worst-case, in-place.

Common mistakes:

- Getting comparator direction wrong in C++ (`greater<T>` for min heap, not `less`).
- In Python, remember `heapq` is a min heap — negate values for max heap.
- Dijkstra: always skip outdated entries from the PQ (lazy deletion).
- Two heaps for median: maintain `left.size() == right.size()` or `left.size() == right.size() + 1.`

17. Revision Notes

Key idea: Heap = complete binary tree where parent $\leq$ (min) or $\geq$ (max) children. Array storage: children at $2i+1, 2i+2$; parent at $(i-1)/2$.

Min heap vs Max heap:

- Min heap: `priority_queue<int, vector<int>, greater<int>>` / `heapq` (default)
- Max heap: `priority_queue<int>` / negate values in Python

Operations (all $O(\log n)$ except top which is $O(1)$):

- `push(x)` — add to end, bubble up
- `pop()` — swap root with end, bubble down
- `top()` — root

Build heap in $O(n)$: start from `n/2 - 1`, bubble down each.

Common patterns:

- **K largest:** min heap of size K
- **K smallest:** max heap of size K
- **Running median:** max heap (left) + min heap (right), size invariant
- **Top K frequent:** hash map + min heap of size K by frequency
- **Merge K sorted:** min heap of (value, listIndex)
- **Dijkstra:** min heap of (distance, node), skip outdated entries

Complexity cheat sheet:

Topic	Time
Push/Pop	$O(\log n)$
Top	$O(1)$
Build heap	$O(n)$
Heap sort	$O(n \log n)$
K largest/smallest (heap of size K)	$O(n \log K)$
Merge K sorted (N total)	$O(N \log K)$
Two heaps (median)	$O(\log n)$ per insertion

Traps:

- C++: `greater<T>` for min heap (not `less`)
- Python: `heapq` is min heap; max heap = negate values
- Dijkstra: `if (d != dist[u]) continue;` — always skip outdated
- Build heap: start from $n/2 - 1$, not $n-1$
- Fibonacci/pairing: know they exist, don't implement in interview

18. Final Cheat Sheet

When to Use Heap / Priority Queue

Problem Pattern	Data Structure
Need min or max of dynamic set	Min heap or max heap
Kth largest / smallest	Heap of size K
Running median	Two heaps (max + min)
Merge K sorted streams	Min heap
Top K by frequency	Hash map + min heap
Shortest path (Dijkstra)	Min heap of (dist, node)
Schedule / reorder tasks	Max heap by count
K closest points	Max heap of size K

Main Operations

Operation	C++ <code>priority_queue</code>	Python <code>heapq</code>
Min heap	<code>greater<int></code>	<code>heapq</code> (default)
Max heap	default	negate values

Operation	C++ <code>priority_queue</code>	Python <code>heapq</code>
Insert	<code>pq.push(x)</code>	<code>heapq.heappush(h, x)</code>
Remove top	<code>pq.pop()</code>	<code>heapq.heappop(h)</code>
Peek top	<code>pq.top()</code>	<code>h[0]</code>

Complexity at a Glance

Operation	Time	
<code>push()</code>	$O(\log n)$	
<code>pop()</code>	$O(\log n)$	
<code>top()</code>	$O(1)$	
<code>buildHeap()</code>	$O(n)$	(Floyd's method)
<code>heapSort()</code>	$O(n \log n)$	(in-place)

Key Code Snippets

```
// Min heap
priority_queue<int, vector<int>, greater<int>> pq;

// Max heap
priority_queue<int> pq;

// K largest: min heap of size K
for (int x : nums) {
    pq.push(x);
    if (pq.size() > K) pq.pop();
}
// pq.top() = Kth largest

// Dijkstra
priority_queue<pii, vector<pii>, greater<pii>> pq;
pq.push({0, src});
while (!pq.empty()) {
    auto [d, u] = pq.top(); pq.pop();
    if (d != dist[u]) continue; // SKIP OUTDATED
    for (auto [v, w] : adj[u])
        if (dist[u] + w < dist[v])
            pq.push({dist[v] = dist[u] + w, v});
}
```

```
# Min heap
import heapq
h = []
heapq.heappush(h, x)
heapq.heappop(h) # smallest
```

```
# Max heap: negate
heapq.heappush(h, -x)
-heapq.heappop(h) # largest

# K largest: min heap of size K
for x in nums:
    heapq.heappush(h, x)
    if len(h) > K:
        heapq.heappop(h)

# Top K frequent
freq = Counter(nums)
for val, cnt in freq.items():
    heapq.heappush(h, (cnt, val))
    if len(h) > K:
        heapq.heappop(h)
```

Important Edge Cases

- **Empty heap:** check before `top()` / `pop()`
- **K = 0 or K > n:** handle explicitly
- **All equal elements:** heap works fine
- **Duplicates:** naturally handled
- **INF values:** use `LLONG_MAX` or `10**18` for safety
- **Directed vs undirected:** Dijkstra adjacency varies

End of Guide. Practice the problems listed in Section 15 and review this document before every interview or coding assessment.